🧠 PayShield: Hackathon Q&A & Judge Survival Guide

Judges love to probe for weaknesses, especially in cybersecurity and ML projects. Here are the most likely "gotcha" questions they will ask, exactly how to answer them without getting stuck, and psychological tactics to leave a lasting impression.

🎯 Section 1: The "Gotcha" Questions (And How to Destroy Them)

Q1: "If an attacker changes the payload *before* your Javascript encrypts or hashes it, how do you catch it?"

Why they ask this: This is the classic "DOM tampering" argument. They are testing if you understand that everything on the client-side can be manipulated. **How to answer:**

> *"That's exactly why PayShield is an **edge-based middleware**, not a client-side obfuscator. We assume the browser is entirely compromised. Attackers use proxy tools to intercept requests precisely because they need to modify the data after the DOM finishes executing but before the server receives it. By inspecting for proxy signatures and execution headers on the Gateway, we catch the tampering in transit, where the attacker is forced to operate."*

Q2: "Why use XGBoost instead of a Deep Learning model or a simple rule-based engine?"

Why they ask this: They want to know you didn't just pick "AI" arbitrarily to get buzzword points. **How to answer:**

> *"Two reasons: Latency and Explainability. Deep Learning models are overkill for tabular financial data like ours and increase latency. In payments, every millisecond counts. XGBoost gives us sub-50ms inference times. Furthermore, XGBoost provides feature importance—meaning our Replay Engine can actually tell the analyst why a transaction was flagged (e.g., 'Distance from last IP was too high'), which Deep Learning 'black boxes' cannot do."*

Q3: "What prevents a legitimate user acting behind a corporate VPN or web proxy from getting flagged as 'tampered'?"

Why they ask this: This tests your understanding of False Positives. **How to answer:**

> *"Great question. We specifically target signatures characteristic of manipulation tools—like Requestly or Burp Suite injected headers (`X-Requestly-Modified`)—not standard `X-Forwarded-For` proxy headers used by corporate VPNs. Additionally, if an anomalous header is flagged, it triggers a step-up authentication or passes the context to our Fraud Engine, which looks at the total risk score rather than blindly blocking every proxy."*

Q4: "Isn't checking headers easy to bypass? What if the attacker just avoids injecting a header?"

Why they ask this: They are poking at the simplicity of the Requestly demonstration. **How to answer:**

> *"Yes, a sophisticated attacker could scrub their own headers. That's why the header check is just **Layer 1** of our defense. If they bypass the middleware, the request still hits **Layer 2: The ML Fraud Engine**. An attacker might spoof the amount, but they can't spoof their historical velocity, geographical mismatch, or card BIN risk simultaneously. Our architecture is defense-in-depth."*

Q5: "How does your system handle scale? What happens during a flash sale with 10,000 requests per second?"

Why they ask this: Testing your architecture choices. **How to answer:**

> *"We decoupled the architecture specifically for scale. The Node.js Express Gateway handles high-throughput asynchronous connections effortlessly. We offload all real-time event broadcasting to **Redis Pub/Sub** and manage state (like rate limits and idempotency) entirely in-memory via Redis. The ML engine is an isolated FastAPI container that can be horizontally scaled independently of the Gateway. It's built for Kubernetes."*

💥 Section 2: How to Impress Them (Psychological Tactics)

1. The "Admit the Limitation" Tactic

Judges respect honesty more than perfection. If they ask about something you haven't built (e.g., "Do you handle zero-day botnets?"), say:

> *"In this 24-hour scope, no. Right now, our focus is entirely on Man-in-the-Middle API tampering. However, the architecture is designed so that a bot-detection service could be plugged directly into the Gateway middleware pipeline as a new microservice without altering the core flow."*

2. Drive Them Back to the Replay Engine

Whenever you explain a complex concept, physically point to the Replay Engine in your dashboard.

> *"To answer that, let me show you what happened on Step 4 of this Replay..." Visualizing your answer proves the system is actually working and gives them eye candy to focus on.*

3. Mention "Idempotency"

Casually drop that your Gateway implements **Idempotency Keys via Redis**.

> *"We also ensure that if a user double-clicks the 'Pay' button, or an attacker replays the exact same request, the Redis idempotency cache catches it instantly and prevents double-charging." This is a senior-level engineering concept that 90% of hackathon teams forget about.*

4. Talk About "Latency Budgets"

Don't just say "it's fast." Say:

> *"We had a strict latency budget of 100ms for the entire security pipeline. The middleware takes 2ms, and the ML inference takes 45ms. We process security asynchronously so the user experience isn't degraded." This shows you think like a real production engineer.*

5. Control the "Requestly" Narrative

Make sure they understand Requestly is your **testing and simulation tool**, not your dependency. We use Requestly to *prove* the threat is real and that our Gateway can catch it. It’s a mechanism for live penetration testing.

⚡ Quick Cheat Sheet (Keep this in your head)

- **Our Enemy:** Client-Side API Tampering (MITM).
- **Our Weapon:** Express Middleware + XGBoost ML.
- **Our Speed:** < 50ms total validation.
- **Our Flex:** We attack our own system live during the demo.
- **Our Architecture:** Fully decoupled, Dockerized Microservices using Redis Pub/Sub.